Data Structures and Amortized Analysis — Worked Solutions

A04-E01 Transfer only when T is empty

Operation	S (bottom→top)	T (bottom→top)	Return	c	Φ	$\hat{c}$
ENQ a	a	empty	—	1	2	3
ENQ b	a,b	empty	—	1	4	3
DEQ	empty	b	a	5	0	1
ENQ c	c	b	—	1	2	3
DEQ	c	empty	b	1	2	1
DEQ	empty	empty	c	3	0	1

Both totals equal 12. Step 3 transfers two records, costing $2 \cdot 2 + 1 = 5$; step 6 transfers one, costing 3. Incorrectly transferring c in step 5 puts it above b in T and returns c first, violating FIFO.

A04-E02 How large must the potential coefficient be?

ENQUEUE has $c = 1$ and $\Delta\Phi = a$, so its charge is $1 + a$. An ordinary dequeue leaves $|S|$ unchanged and costs 1. A transfer gives

$$\hat{c} = (2k + 1) - ak = 1 + (2 - a)k.$$

For $a < 2$, this grows without bound with k. For fixed $a \geq 2$, it is at most 1, while enqueue costs the constant $1 + a$. Thus $a \geq 2$ is necessary and sufficient. Values above 2 can give negative transfer charges; the nonnegative potential and zero initial potential still validate the ledger. Setting a=2 makes every successful dequeue charge exactly 1.

A04-E03 A nonempty initial state is not free

Four pops from S, four pushes onto T, and the final pop cost $c = 2 \cdot 4 + 1 = 9$. Initial potential is 8 and final potential is 0, so $\Delta\Phi = -8$ and the charge is 1.

$$c = \hat{c} + \Phi_0 - \Phi_1 = 1 + 8 - 0 = 9.$$

The initial eight units of transfer credit cannot be omitted. Including four earlier enqueues from an empty structure gives actual cost $4 + 9 = 13$ and total charge $4 \cdot 3 + 1 = 13$, with zero potentials at both endpoints.

A04-E04 A complete growth-and-contraction ledger

Operation	(n, M)	Moves	c	Φ	$\hat{c}$
Append a	(1,1)	0	1	1	2
Append b	(2,2)	1	2	2	3
Append c	(3,4)	2	3	2	3
Append d	(4,4)	0	1	4	3
Append e	(5,8)	4	5	2	3
Delete e	(4,8)	0	1	0	-1
Delete d	(3,8)	0	1	1	2
Delete c	(2,4)	2	3	0	2
Delete b	(1,2)	1	2	0	2
Delete a	(0,1)	0	1	0	1

Ten moves and ten successful operations give actual cost 20; charges also total 20. Deleting e releases two units of potential while performing one unit of work, producing a charge of -1, not negative running time. The final 2→1 contraction moves nothing because the table is empty.

A04-E05 A counterexample to shrinking at half occupancy

The first eight insertions cost $8 + (1 + 2 + 4) = 15$ and reach (8,8). Each pair cycles as

$$(8, 8) \xrightarrow[\text{move } 8]{\text{APPEND, } c=9} (9, 16) \xrightarrow[\text{move } 8]{\text{DELETE, } c=9} (8, 8).$$

Four pairs cost 72 more, totaling 87 over $m = 16$ operations; $87 > 3m = 48$. This refutes transplanting the stated constant bound to another resize rule. One numerical example alone is not an asymptotic counterexample.

For powers of two k, fill k slots in O(k) work and perform k pairs, each costing 2k+2. The 3k operations incur at least $2k^2 + 2k$ work, ruling out any fixed constant amortized upper bound for this family.

A04-E06 Does adding a constant to potential break the proof?

The claim is false. The change is unchanged: $\Delta\Psi_i = (\Phi_i + 7) - (\Phi_{i-1} + 7) = \Delta\Phi_i$, so every charge is identical.

$$\sum c_i = \sum \hat{c}_i + \Psi_0 - \Psi_m = \sum \hat{c}_i - \Phi_m \leq \sum \hat{c}_i.$$

Although $\Psi_0 = 7$, every reachable endpoint has $\Psi_m \geq 7 = \Psi_0$. The endpoint condition is what matters; zero initial potential and nonnegativity are convenient sufficient conditions.

A04-E07 Constant amortized cost is not constant latency

After n enqueues, S contains n records and T is empty. The next dequeue costs 2n+1 stack primitives. A full array at (M,M) requires M+1 units for its next append. Both states are reachable from empty.

A total-cost bound permits concentrated work in one call and provides no per-call worst-case latency bound. A fixed budget requires a representation with such a bound, or incremental migration with correct queries and updates while old and new representations coexist. That migration needs its own invariants; saying the structure is fast on average does not establish the missing guarantee.

A04-E08 Middle deletion is outside the original theorem

The deletions shift $n-1, n-2, \ldots, 0$ records, totaling $n(n-1)/2$ moves. Adding n deletion units and O(n) construction work gives $\Theta(n^2)$ total.

There are only 2n operations, so this operation set does not have constant amortized cost. The theorem covers APPEND / DELETE-LAST without suffix shifts; infrequent reallocation does not eliminate the linear shift performed by each front or middle deletion.

A04-E09 A four-bit wraparound ledger

There are three initial set bits, so $\Phi_0 = 3$.

Result	c	Φ	$\Delta\Phi$	$\hat{c}$
1110	2	3	0	2
1111	1	4	1	2
0000	4	0	-4	0

Actual cost is 7 and amortized cost is 4. Telescoping gives $7 = 4 + 3 - 0$. Charges at most 2 do not imply actual cost at most $2m$ from a nonzero initial state. Include initial potential, giving an upper bound of 9 here. Wraparound clears four bits without setting a fifth bit.

A04-E10 Charging units for MULTIPOP and MULTIPUSH

Actual costs are $1, 1, 1, 2, 1$ and balances are $1, 2, 3, 1, 0$. Every prefix stays solvent. Total charge and actual cost both equal 6. The final request removes only a, despite its argument 9.

MULTIPUSH(k) must write k items, costing at least k. One such call from empty refutes a constant charge independent of k. Charging 2 per inserted item gives this call a charge of $2k$. Bound total work by inserted-item count plus call count; variable-length input is not free.

A04-E11 An explicit increment/decrement counterexample

Preparation flips $1, 2, 1, 3, 1, 2, 1$ bits, totaling 11, and ends at 0111. Each pair goes 0111→1000→0111, flipping four bits per call and costing 8. There are 15 calls and total actual cost $11 + 4 \cdot 8 = 43$.

For k bits, prepare with $r = 2^{k-1} - 1$ increments and perform r pairs, each costing $2k$. With $m = 3r$ calls, cost is at least $2rk = (2/3)mk$ and at most mk, hence $\Theta(mk)$ on this family. The fixed-width numerical case only checks arithmetic; disproving a constant bound independent of k requires allowing k to grow.

A04-E12 Does a rejected enqueue preserve FIFO?

Here — marks a nonmember, not a required physical clear.

Call	Physical A[0..2]	h	s	Return
ENQ A	[A,—,—]	0	1	success
ENQ B	[A,B,—]	0	2	success
ENQ C	[A,B,C]	0	3	success
ENQ D	[A,B,C]	0	3	overflow
DEQ	[—,B,C]	1	2	A
ENQ E	[E,B,C]	1	3	success
DEQ	[E,—,C]	2	2	B
DEQ	[E,—,—]	0	1	C
DEQ	[—,—,—]	1	0	E

A rejected insertion preserves state. Advancing h would lose the original front A. The contract rejects overflow rather than overwriting the oldest ring-buffer item.

A04-E13 Why reversal saves nxt first

Initially prev=NIL and cur=a→b→c. After iteration 1, prev=a→NIL and cur=b→c. After iteration 2, prev=b→a→NIL and cur=c. After iteration 3, prev=c→b→a→NIL and cur=NIL; assign head=prev.

The reversed processed prefix and untouched suffix are disjoint and together contain every original node. Reading cur.next after rewiring makes cur=NIL in the first iteration, since a.next was just cleared. The algorithm loses its saved route to b and c. Save nxt before overwriting the link.

A04-E14 A sentinel list with one data item

With p=q=z, insertion writes x.prev=z, x.next=z, z.prev=x, z.next=x. Both directions from z reach x, and both links from x return to z.

Deletion saves p=q=z and writes z.next=z and z.prev=z, restoring empty. The sentinel is a permanent structural anchor, not data. Reclaiming it invalidates the empty representation, search termination, and later operations. The deletion contract requires a valid data member $x \neq z$.

A04-E15 Does a recycled handle still identify the old object?

Allocating 1 leaves free=4. After appending D, head=5 and live is 5(A)→2(B)→3(C)→1(D)→0. Removing B writes next[5]=3 and prev[3]=5; FREE(2) writes next[2]=4 and free=2. Final live is 5→3→1→0 and free is 2→4→0. They partition all five slots.

The next allocation returns 2 and leaves free=4. The caller temporarily owns that unlinked slot. The old in-range handle no longer denotes B. Callers must honor invalidation; detecting stale handles could use checked generation numbers, an extra mechanism not supplied by plain indices.

A04-E16 Wraparound, deletion, and updating a duplicate

Call	Slots	Result
INSERT(4,a)	4	store at 4
INSERT(9,b)	4,0	store at 0
INSERT(14,c)	4,0,1	store at 1
DELETE(9)	4,0	mark 0 DELETED
INSERT(14,c′)	4,0,1	update 1
INSERT(19,d)	4,0,1,2	store (19,d) at 0
FIND(24)	4,0,1,2	EMPTY at 2: absent

Final slots 0–4 are [(19,d),(14,c′),EMPTY,EMPTY,(4,a)], with n=3. The tombstone at 0 precedes existing key 14 at 1, so inserting immediately duplicates it. New key 19 first establishes absence at 2, then reuses 0.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A04-E17 Enumerate one affine family

Compute r=(a+b) mod5 and s=(4a+b) mod5, then reduce modulo 3. Distinct residues collide only for ordered pairs (0,3),(3,0),(1,4),(4,1).

a	Colliding b
1	0,4
2	none
3	none
4	0,4

Four of 20 parameter choices collide, giving $1/5 \leq 1/3$ and satisfying the "at most" definition. This checks one pair only; universality requires the bound for every distinct fixed pair, as established by the general proof.

A04-E18 The cost of a noncoprime step

Step 2 visits 1,3,5,7, then returns to 1. Its period is 8/gcd(8,2)=4. Slot 0 is unreachable; eight probes merely repeat those four slots.

Step 3 visits 1,4,7,2,5,0,3,6 with period 8. Triangular offsets are 0,1,3,6,2,7,5,4, another permutation. Divide the even integer i(i+1) by 2 before reducing modulo 8. There is no inverse of 2 modulo 8, so modular division by an inverse is invalid.

A04-E19 An exact expectation versus its upper bound

The tails are 1,2/4=1/2, and (2/4)(1/3)=1/6, then zero. Thus E[X]=1+1/2+1/6=5/3, below the bound 1/(1−1/2)=2.

After probing one occupied slot, only one of three remaining slots is occupied. Its conditional probability is 1/3, not 1/2. This samples a permutation without replacement. α^2 is an upper bound here, not the exact probability.

A04-E20 FKS space and membership

Bucket sizes [0,1,2,2,1,0] give S=10. The two size-two buckets each contribute one collision pair, so Y=2 and n+2Y=6+4=10<24.

Query 26 reaches bucket 2, secondary slot 2, and reads 26: present. Query 14 follows the same route but reads 26: absent. Query 5 reaches empty bucket 5 and stops. Query 10 reaches singleton bucket 4 but reads 4: absent.

Skipping key equality falsely accepts 14 and 10. Check the square sum and collision freedom separately: one bounds space, the other separates stored keys. Neither replaces query equality testing.

A04-E21 Why deleting everything need not make search cheap

Here n=0,d=5, so α=0 and β=1. FIND(7) starts at 2 and probes 2,3,4,0,1. With no EMPTY, it returns absent only after all five probes.

There is no contradiction: the theorem assumes no tombstones and ideal random permutations, neither satisfied here. Any full-table permutation still scans all five DELETED slots. Live load α is not the stopping probability. Rebuilding to all EMPTY permits one-probe failure, but rebuilding costs extra work.

A04-E22 Which step fails for a composite modulus?

h_a(0)=0, while h_a(2) is 0,2,0,2 for a=0,1,2,3. Two choices collide, giving 1/2>1/4. This family on that universe is not universal under our definition.

The nonzero difference 2 has no inverse modulo 4. Equation 2a=0 has two solutions rather than one. The failed step is invertibility, not linearity of expectation. This refutes this construction, not all universal families with composite table sizes; the affine family allows arbitrary m.

A04-E23 Local comparisons can miss an ancestor violation

Node 25 belongs to 20’ s left subtree and 10’ s right subtree, requiring 25∈(10,20), which is false. FIND(25) visits 20→30→NIL and reports absent. The representation violates the invariant; this does not refute search correctness on valid BSTs.

A04-E24 Same tree, different query distributions

Successful comparisons are 1,2,2,3,3, averaging 11/5. Failed queries 1,4,6,9,10 cost 2,3,3,2,2, totaling 12; the mixed mean is 23/10. Query 9 alone follows 5→8→NIL and costs 2. The query distribution is part of any average-cost claim.

A04-E25 Two routes to the successor

For 5, descend to 8 and then left to 7. For 3, climb to 2 because 3 is its right child; then reach 5 from its left side, returning 5. For 8, climb through 5 from its right side and then reach NIL. A larger ancestor can be the successor when no right subtree exists.

A04-E26 An immediate successor must skip the deeper-successor branch

Since z.right is y, the assignment creates y.right=y, a cycle, and overwrites the link to 50. Skip that branch. Set root=y,y.parent=NIL,y.left=10,10.parent=y; retain y.right=50 and 50.parent=y. Only object 20 leaves the tree.

A04-E27 Root deletion when the successor has a right child

The successor is 25 with right child 27.

Step	Writes
Replace the successor at its old position	30.left=27;27.parent=30
Attach the old right subtree	25.right=40;40.parent=25
Replace the root	root=25;25.parent=NIL
Attach the old left subtree	25.left=10;10.parent=25

Other links remain. Inorder becomes 10,25,27,30,40,50. Transplanting invalidates original object 20 while preserving object 25 at a new location. Copying instead changes object 20’ s key/value and invalidates original object 25. Copying only the key would mismatch the dictionary record.

A04-E28 Expected height over six insertion orders

Orders 213 and 231 have height 1; 123,132,312,321 have height 2. Thus $E[X_3]=5/3$ and $E[2^{X_3}]=10/3$; $2^{(5/3)}\approx3.175<10/3$. Uniformly sampling the five shapes instead gives $(1+4\cdot2)/5=9/5$, a different model.

A04-E29 Fixed-rank versus uniform-rank expected search cost

$H_3=11/6$ gives $E[D_3]=2H_3-2=5/3$ and expected successful comparisons 8/3. With $H_5=137/60$, expected total depth is $12 \cdot 137/60-20=37/5$. Uniform successful queries therefore have expected mean$(37/5)/5+1=62/25$. A fixed rank and a uniformly selected stored key are different experiments.

A04-E30 Bases and inequalities in the exponential-height proof

$Y_0=0$ is an auxiliary convention; define $Y_n=2^{X_n}$ only for $n\geq 1$, verify $Y_1=1$, and start the recurrence at $n\geq 2$. Expectation cannot pass through max. Condition on root rank and use $\max(u,v)\leq u+v$ to obtain $a_n\leq(4/n)\sum_{j=0}^{n-1}a_j$. Induction and the cube sum give $a_n\leq(4/n)[n(n-1)/2]^2=n(n-1)^2\leq n^3$. Convexity gives $2^{E[X_n]}\leq E[2^{X_n}]=a_n\leq n^3$. Taking $\log_2$ gives $E[X_n]\leq 3\log_2 n$; individual orders may still produce chains of height $n-1$.

A04-E31 • Black-height versus full-path black count

Initially bh(20)=1, bh(10)=1, bh(NIL)=0. A full path counts 20 and NIL, giving 2 blacks. After recoloring, bh(20)=2, bh(10)=1, bh(NIL)=0; a full path counts 20, either child, and NIL, giving 3. Black-height excludes its starting node.

A04-E32 • Outside parent and an empty middle subtree

Write 10.right=15, 15.parent=10, 20.parent=30, 30.left=20, 20.left=10, 10.parent=20. For a root rotation use 20.parent=NIL and T.root=20. Omit the second write for β=NIL to preserve the shared sentinel context. The old parent of y is x; update the old parent p of x to point to y. Inorder 10,15,20 is preserved, followed by 30 if the outside parent is included; object identities are unchanged.

A04-E33 • Black counts for non-root I1

Before: bh(g)=b and the count including g is 1+b. After: bh(g)=b+1 but the count including g is still 1+b. Ancestor paths do not gain a black. Only g and its parent may now conflict. If g is the root, final blackening leaves bh(g)=b+1 but raises each full-root path to b+2, preserving equality.

A04-E34 • I2 cannot return early

After insertion root 35B has left 15R with right 25R; z=25. I2 sets z=15 and left-rotates 15, producing 35.left=25R and 25.left=15R. Parents are NIL,35,25; the red pair 15/25 remains. I3 blackens 25, reddens 35 and right-rotates 35: root 25B has children 15R,35R, both parent links 25, and root.parent=NIL; z remains 15. For 15,35,25 first right-rotate 35 with z=35, then blacken 25, redden 15 and left-rotate 15. Final full paths contain 2 blacks; at most 2 primitive rotations occur.

A04-E35 • Counting boundaries in the height proof

For NIL,0=2⁰−1. Each internal node's children have black-height at least bh(x)−1, so induction gives

$$n_x \geq 2(2^{bh(x)-1} - 1) + 1 = 2^{bh(x)} - 1.$$

On a longest path with B blacks and R reds, black endpoints and no consecutive reds imply R≤B−1. Thus H=B+R−1≤2(B−1)=2bh(root). Combine with $n \geq 2^{H/2} - 1$ and take logarithms. Empty: h=−1,H=0. Singleton: h=0,H=1. At n=0,log(n+1)=0 cannot cover a constant test; use O(1+log(n+1)) or O(log(n+2)).

A04-E36 • Compare repair schedules precisely

Only I1 propagates upward in CLRS bottom-up repair, and it performs no rotation. I2/I3 use at most two primitive rotations and terminate. Total rotations therefore have the stronger constant bound 2; O(log(n+2)) time comes from search and upward recoloring. Top-down repair splits 4-nodes during descent; CLRS repairs after inserting a red leaf. Their entry invariants and recoloring schedules differ and may produce different valid shapes. One double-rotation adjustment consists of two primitive rotations.

A04-E37 • Original versus relocated color

z=40B, y=45R, x=NIL. Set 50.left=NIL and NIL.parent=50; set 45.right=60 and 60.parent=45; set root=45 and 45.parent=NIL; attach 45.left=20 and 20.parent=45; finally set 45.color=B. No black was removed from the old successor position, so no fixup is needed. The new black color replaces z's contribution, not y's old contribution. Only object 40 leaves; reset NIL.parent only after the operation.

A04-E38 • Last-node deletion and red-child absorption

For a singleton, transplant gives root=x=NIL, so the loop is skipped. Final blackening preserves the black sentinel; the empty tree is valid. In the second case root=x=15R, so the loop is again skipped; blackening 15 absorbs the removed black. Stored colors remain R/B. The cursor and proof ledger represent the extra contribution; never recolor NIL red or delete NIL as a data record.

A04-E39 • Why D1 must continue into D2

After removal x=NIL=25.left, p=25B, w=45R. D1 blackens 45, reddens 25 and left-rotates 25, making root 45. The deficit stays at 25.left; refresh w to 35B. Its NIL children are black, so D2 reddens 35 and returns p=25R. The outer loop exits on red x and blackens 25. Final tree:45B with left 25B(right 35R) and right 55B(right 65R). One rotation occurs; final blackening of 25, not D1 alone, absorbs the extra black.

A04-E40 • Path counts through D3 and D4

Before D3 the left path has actual 2+extra 1=3; the right path has actual 3+extra 0=3. D3 blackens 35, reddens 45 and right-rotates 45: root 25 retains its deficient left NIL and has right 35B(right 45R). The left total remains 2+1=3; both right paths contain 3 actual blacks. D4 gives 35 the old parent color, blackens 25 and 45, and left-rotates 25. Final tree:35B with 25B,45B; every full path has 3 actual blacks and no extra. D3 only converts the case; D4 finishes it. Two primitive rotations occur.

A04-E41 • Right deficit and an immediate successor

z=35B and immediate successor y=45B has x=y.right=NIL. Set NIL.parent=45 before relocating y; afterward x is 45.right. Left sibling w=15B has near=w.right=25R and far=w.left=NIL. Mirrored D3 blackens 25, reddens 15 and left-rotates 15; refresh w=25B. Mirrored D4 gives 25 the old color of 45, blackens 45 and far 15, then right-rotates 45. Final root 25B has 15B,45B. Treating left as near inspects the wrong nephew; retaining parent 35 would dereference the removed object.

A04-E42 • Why deletion has at most three rotations

D2 performs no rotation and moves the unique extra contribution one level upward. A red parent absorbs it; a root removes it globally; otherwise repair repeats. D1 makes the old parent red: a following D2 is absorbed there, while any other path reaches D4 directly or via D3 and terminates. Thus no later D1 can follow a D1. D3 immediately enters D4, which removes the extra contribution and returns the root. Each rotating case occurs at most once, giving at most 3 primitive rotations. Search, successor search, and upward D2 propagation still cost O(log(n+2)).

A04-E43 • Local and global ranks

At 20, k=7 and r=4: move right with k=3. At 40, compare 3 with r=5 and move left without changing k. At 30, r=3, so return 30. For rank(27), start at 1; the right-child step 27→25 adds 1, the next two left-child steps add 0, and 40→20 adds size(10)+1=4, giving 6. Empty trees admit no valid selection. Duplicate keys need a total order such as (key,identity); rank identifies an object position, not a unique position for a repeated value.

A04-E44 • Summaries after successor relocation

Save a=30 before relocation. After all links change, 27 fills 30.left, while new root 25 has left 10 and right 40. Follow final parents 30→40→25, computing 1+1+1=3, 1+3+1=5, and 1+3+5=9. Starting at the old root's parent, NIL, repairs none of these nodes. Pulling 25 first is also wrong because it reads stale 40.size=6.

A04-E45 • A local formula does not imply two-node-only repair

Rotation preserves the object set of the whole local subtree. The promoted y therefore inherits the old x subtree's size/max, leaving ancestor summaries unchanged. After pointer updates, pull(x) before pull(y), because new y depends on new x. Height also has a local formula but can change for the entire rotated subtree and propagate to ancestors. Inserting a new minimum raises every old global rank by one; explicitly storing all those values requires Θ(n) writes.

A04-E46 • A larger max is not necessarily safe

At root [12,17], stale left.max=27≥22 sends the search left. At [5,7], left[2,4].max=4<22 sends it into empty right, incorrectly returning NIL. Right subtree [20,24] is a true overlap. Exact max=7 sends the first step right. The leftward proof requires a witness j in the left subtree whose high equals max. An arbitrary bound of 27 need not have such a witness, so the implication from no left answer to no right answer fails.

A04-E47 • Touching endpoints and field updates

Closed intervals overlap at 5: use two ≤ comparisons and left.max≥q.low. Half-open intervals do not overlap: use two < comparisons and left.max>q.low. [5,5) is empty and must be rejected or handled by an explicit contract. Editing low changes the ordering key and requires deletion/reinsertion. Editing high preserves ordering, but requires a nonempty-interval check and upward max recomputation, including decreases.

A04-E48 • The cost of reporting all answers

There are k successful searches and one final failed search proving exhaustion; that last operation remains when k=0. Using original size n uniformly, searches, deletions, and reinsertion cost O((k+1)log(n+2)), with O(k) result storage and temporary set mutation. This does not prove O(log n+k); a tighter output-sensitive bound needs another algorithm or a separate visited-node analysis.

A04-E49 • Capacity parameters and split timing

For m=3, keys range from ⌈3/2⌉−1=1 to 2. For m=4 they range from 1 to 3, matching t=2. Promoting a key from [2,8] leaves only one key for two children, forcing one empty child and violating the minimum. Overflowing first to [2,5,8] allows promotion of 5 with one key on each side. The exact correspondence is m=2t, not an arbitrary substitution for m.

A04-E50 • Height and page reads for a billion keys

The dense bound $n \leq 200^{h+1} - 1$ excludes h=2 (maximum 7,999,999) and permits h=3 (maximum 1,599,999,999). The sparse bound $n \geq 2 \cdot 100^{h} - 1$ permits h=4 (minimum 199,999,999) but excludes h=5 (minimum 19,999,999,999). Hence h is 3 or 4. A path has h+1 nodes; skipping the resident root yields 3 or 4 reads. A nonresident root yields 4 or 5.

A04-E51 • Every child pointer in an internal split

Promote 20, giving parent[20,50] with children[10],[30],[70]. Node[10] owns[5],[15]; node[30] owns[25],[35]; node[70] still owns[60],[80]. No child retains 20. Inorder remains 5,10,15,20,25,30,35,50,60,70,80. Every leaf remains at depth 2 and each one-key internal node has two children.

A04-E52 • A deletion that borrows and then merges

Move root 20 into left internal node[10,20], raise sibling minimum 30 to the root, and leave right internal node[40]. Subtree[25] must become the last left-child subtree since 20<25<30. Merge[5], separator 10, and[15], then remove 5 to obtain[10,15]. Final root[30] has left node[20] with leaves[10,15],[25], and right node[40] with leaves[35],[45]. Retaining[25] on the right puts 25<30 in root 30's right subtree, violating global order.

A04-E53 • Payload consistency during internal replacement

A uses predecessor 10: delete it on the left and move the saved whole record to the root, leaving root[10] and leaves[5],[30]. B has a minimal left child and a rich right child, so use successor 30, leaving root[30] and leaves[5],[40]. Changing only the key while retaining record 20's payload makes lookup return the wrong data. Save the whole record or a stable record pointer; do not read freed source storage.

A04-E54 • Root repair after a failed deletion

Initially choose child 1. Both leaves are minimal, so merge left and change the target index to 0. Search in[5,20,30] fails and returns NOT_FOUND. The old root now has zero keys and one child and still needs contraction. Final leaf root[5,20,30] represents the unchanged set. Returning early leaves an illegal zero-key internal root. Shape-preserving failure requires an initial existence search.

A04-E55 • Why only the root may contract

Merge into[5,20,30], remove 20, and promote leaf[5,30] over the empty internal root, reducing h from 1 to 0. Removing 5 leaves root[30] at h=0; removing 30 retains a zero-key leaf root, also h=0 by convention. Root contraction shortens every leaf path equally. Contracting an ordinary internal node shortens only some paths and breaks equal depth. Correct pre-descent repair also prevents a nonroot internal node from becoming empty.

A04-E56 • Which costs improve with within-page binary search?

All three operations use O(h+1) page accesses because they handle only constantly many pages per level. Binary-search lookup uses O((h+1)log t) comparisons; linear scanning uses O(t(h+1)). Insertion and deletion may still shift Θ(t) keys and child pointers after locating the slot, so their CPU upper bound remains O(t(h+1)). The analysis assumes a page-capacity, cache, and write model; page counts are not measured latency.

A04-E57 • Circular lists and the meld contract

Save al=a.left and bl=b.left; set al.right=b, b.left=al, bl.right=a, a.left=bl. These four endpoints join the circles without visiting other nodes, giving O(1). Independently operating both the result and old heaps would mutate shared nodes while maintaining inconsistent counts, minima, and lists. Consume the old entries. Empty-list cases choose the other entry without dereferencing NIL.

A04-E58 • Complete consolidation and child promotion

The first links are 9→4, 12→7, 7→4, and 15→6, with arrows from child to parent. Roots are 4(9,7(12)), 6(15), and 10. Removing 4 promotes 9 and 7. Process roots 10(d=0),6(d=1),9(d=0),7(d=1). At 9, link 10→9 and then 9→6, producing degree-two root 6. Root 7 remains at degree one. Final roots are 6 with children 15 and 9(10), and 7 with child 12. Thus min=6 and n=6; sizes 4+2 and heap order agree.

A04-E59 • A decrease moves a subtree, not just a key

For 15→14, 14≥12, so all pointers, degrees, and marks stay unchanged and min=1. For 15→11, cut the whole subtree 11(20): 11.parent=NIL, its mark=false, and 20.parent still names that same node. Node 12 loses one child, so its degree falls 1→0 and its first-loss mark becomes true. Root 1 stays unmarked. Roots are 1 and 11; node 11 still has degree one and min remains 1.

A04-E60 • Root and nonroot grandparent outcomes

If g=2 is a root, cut x, cut and unmark p, then stop at g. Roots are {2,1,8}; p retains 15 and all roots are unmarked. Counts change (1,1)→(3,0), unscaled potential 3→3, and min=1. If q=0 is the root and g=2 an unmarked nonroot, mark g after the same cuts. Roots are {0,1,8}; q retains g, p retains 15, and only g is marked. Counts change (1,1)→(3,1), potential 3→5, and min=0.

A04-E61 • Constants in the potential proof

For c≥1, c roots are added. The last c−1 cuts clear marked nodes, and at most one final mark is added: Δm≤2−c and ΔΦ≤K(4−c). Fix K≥a, giving ĉ≤a(c+1)+K(4−c)≤a+4K. The hidden coefficient in O(c) can exceed one, so unscaled subtraction is insufficient. The same K must also cover extract-min root-processing constants; choose a maximum of these fixed constants, independently of c and the operation count.

A04-E62 • Bound degree without assuming logarithmic height

At attachment, the i−1 earlier surviving children were already present, so the parent had degree at least i−1. Equal-degree linking gives the new child the same lower bound. Losing two children would cut it away; a surviving child loses at most one, yielding i−2. Strong induction on subtree size applies to every child, giving size≥2+$\sum_{i=2}^{d} F_i$=F_{d+2} for d≥2; bases d=0,1 give 1,2. With $F_{d+2} \geq \varphi^d$ and n≥size, $d \leq \log_\varphi n$. This bounds immediate children, not longest paths, and does not prove O(log n) worst-case decrease-key.

A04-E63 • Duplicate minima and deletion identity

If min names u and only a strict decrease updates min, assigning an equal −∞ leaves min=u, so extraction removes u. Compare ordinary keys as (1,key), but temporarily give only x a tag-zero key (0,anything), uniquely below all ordinary keys. Decrease, extract, and restore the removed object’s old key. The shared potential cancels between the two operations, giving O(log(n+1)) amortized cost. Locating x remains a separate cost.

A04-E64 • The actual cost of deferred work

The n insertions leave n singleton roots and potential Kn. Extraction processes the remaining n−1 roots, taking Θ(n). Consolidation leaves only O(log(n+1)) roots and never adds marks, releasing potential accumulated by insertions to pay for root scans and links. Constant amortized insertion charges already saved this work; total sequence cost is O(n+log(n+1)). For a general initial state, $\sum_i c_i = \sum_i \hat{c}_i + \Phi_0 - \Phi_{\text{final}} \le \sum_i \hat{c}_i + \Phi_0$; a large initial potential cannot be discarded.

A04-E65 • Odd widths and local coordinates

Top level: m=5,H=8,L=4. Key 21 splits(5,1) and 30 splits(7,2), reconstructing 21 and 30. Offset 1 enters C_5 with u=4: m=2,H=L=2, so 1 splits(0,1) and reconstructs 1. Its u=2 child is the base. The top-level summary has universe 8, not 4.

A04-E66 • Reconstruct the set and trace three successors

Extrema are 2 and 30. C_0={3},C_1={3},C_4={2},C_5={1},C_7={2}; summary={0,1,4,5,7}. Query 3 has i=0,j=3, so the current cluster has no larger offset; the summary yields 1, whose cluster min 3 reconstructs 7. Query 20 has i=5,j=0<1, so recurse in C_5 and reconstruct 21. Query 30 meets x≥max and returns NIL; successor is strict.

A04-E67 • Predecessor is not a perfectly symmetric copy

predecessor(3)=2: C_0 has no smaller offset and the summary has no index below 0, but detached min 2 is smaller than 3. predecessor(2)=NIL because x≤min. predecessor(31)=30 because x>max and 31 is still within the universe. The symmetric-but-incomplete version incorrectly returns NIL for query 3.

A04-E68 • Swap out min and activate a new cluster

Insert 1 swaps with min 2. Old 2 enters C_0, changing its local set{3} to{2,3}; summary remains{0,1,4,5,7}, min=1,max=30. Key 10 splits i=2,j=2. Empty C_2 requires inserting index 2 into the summary and setting C_2.min=C_2.max=2. Summary becomes{0,1,2,4,5,7}. Initializing C_2 is O(1); only the summary can recurse expensively. Inserting 1 instead recursed into nonempty C_0.

A04-E69 • Promote min and reduce to a singleton

Both read summary.min=0 and C_0.min=3, then set x=min=3 and delete local offset 3 from C_0. Singleton C_0 empties immediately; remove index 0 from the summary. For S, summary becomes{1,4,5,7} and x=3 differs from max 30. For T the summary empties and x equals old max 3, so max=min=3. The promoted minimum has no remaining cluster copy.

A04-E70 • The actual cost of two recursive calls

A summary deletion occurs only when the cluster empties. That cluster was a singleton, making its deletion O(1). If it had at least two keys, it remains nonempty and no summary deletion occurs. Thus $R(m) \le R(\lceil m/2 \rceil) + c$ with R(1)=O(1). Since ceil(m/2)≤2 m/3 for m≥2, depth is O(log m), giving $O(1 + \log\log u)$ worst-case time. No amortized cancellation is involved.

A04-E71 • Space induction for full allocation

Choose B≥c and A≥B+P(2), establishing P(2)≤A−B. Substitution yields $P(u) \le A[H(L-1) + H - 1] - B(H+1) + cH \le A(u-1) - B$. Thus space is O(u); cluster-only base nodes already require Ω(u), giving Θ(u). Allocating and initializing all nodes takes Θ(u), so q operations cost $O(u + q(1 + \log\log u))$ in total.

A04-E72 • What invalid update preconditions can destroy

DELETE-PRESENT sees min=max=1 and clears both, wrongly erasing 1. INSERT-ABSENT(1) stores its offset in C_0 while top-level min stays 1, duplicating the detached minimum below. Public wrappers validate 0≤x<u, call MEMBER, and update only when the respective precondition holds. Otherwise return a clear status without mutation. The check plus update remains worst-case O(1+log log u).

References and attribution

Cormen, Leiserson, Rivest, Stein. Introduction to Algorithms, 3rd ed. (2009), Chapters 10–14, 17–20. MIT 6.046J/18.401J (2004), L09: Balanced Search Trees, Piotr Indyk. MIT 6.046J/18.401J (2005), L08: Hashing II; L09: Randomly Built BSTs; L11: Augmenting Data Structures; L13: Amortized Analysis. Charles E. Leiserson and Erik D. Demaine. Lin Cing, personal handwritten data-structure notes: queues, search trees, red-black trees, multiway search trees, and heaps.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study